

② 算术运算类指令

这组指令的操作数可以是8位、16位，当存储单元是该类指令操作数时，操作数的寻址方式可以是任意一种存储单元寻址方式

加法指令: ADD ADC INC
减法指令: SUB SBB DEC NEG CMP
乘法指令: MUL IMUL
除法指令: DIV IDIV
符号扩展指令: CBW CWD
十进制调整指令: DAA DAS AAA AAS AAM AAD

用来执行二进制算术运算(加、减、乘、除)，会根据运算结果影响状态标志

(1) 加法指令

1) 普通

ADD 目的操作数, 源操作数
(源) + (目的) → (目的)

2) 带进位

ADC 目的操作数, 源操作数 ^{CF}
(源) + (目的) + 低位进位 → (目的)

3) 加1

INC 目的操作数
(目的) + 1 → (目的)

仅INC指令不影响CF标志, 其余均对条件标志位有影响

条件标志位

⇒ 状态标志:

SF (符号标志)

| 表示结果为负

ZF (零标志)

| 表示结果为零

无符号数相加溢出 → CF (进位标志)

| 表示和的最高有效位有向高位的进位

有符号数相加溢出 → OF (溢出标志)

| 表示两操作数符号相同但结果符号相反

PF (奇偶标志)

| 表示结果低8位中有偶数个1

AF (辅助进位标志)

| 表示低4位向高4位有进1借位

⇒ 控制标志:

DF (方向标志)

| 表示字符串操作从高地址向低地址进行(反向)

IF (中断允许标志)

| 表示允许响应可屏蔽中断

TF (陷阱标志)

| 表示进入单步调试, 执行一条指令就产生一次中断

例.

$n=8\text{ bit}$ 有符号数 $(-128 \sim +127)$, 无符号数 $(0 \sim 255)$

$$\begin{array}{r} 0000\ 0100 \\ +\ 0000\ 1011 \\ \hline 0000\ 1111 \end{array}$$

有: $OF=0$ $(+4) + (+11) \stackrel{?}{=} +15 \checkmark$

无: $CF=0$ $4 + 11 \stackrel{?}{=} 15 \checkmark$

都不溢出

$$\begin{array}{r} 1000\ 0111 \xrightarrow{\text{除符号取反}} 1111\ 0100 \rightarrow -0001011 = -11 \\ +\ 1111\ 0101 \xrightarrow{\quad} \\ \hline 0111\ 1100 \end{array}$$

有: $OF=1$ $(-12) + (-11) \stackrel{?}{=} +134 \times$

无: $CF=1$ $135 + 245 \stackrel{?}{=} 124 \times$

都溢出

$$\begin{array}{r} 0000\ 0111 \\ +\ 1111\ 1011 \rightarrow 1111010 \rightarrow -0000101 = -5 \\ \hline 0000\ 0010 \end{array}$$

有: $OF=0$ $(+7) + (-5) \stackrel{?}{=} +2$

无: $CF=1$ $7 + 251 = 258$

仅无符号数溢出

$$\begin{array}{r} 00001001 \\ +\ 01111100 \\ \hline 10000101 \end{array}$$

有: $OF=1$ $(+9) + (+124) \stackrel{?}{=} +133$

无: $CF=0$ $9 + 124 = 133$

仅有符号数溢出

例. 双精度数的加法

$(DX) = 0002H$ $(AX) = 0F365H$ $(BX) = 0005H$ $(CX) = 8100H$

指令序列 $ADD\ AX, CX(1)$

$ADC\ DX, BX(2)$

(1) 执行后:

$$\begin{array}{r} 0111 \\ 0F365H \\ +\ 00008100H \\ \hline 011107465H \end{array}$$

$OF=1, CF=1, SF=0, ZF=0$

(2) 执行后:

$$\begin{array}{r} 0005H \\ +\ 0002H \\ \hline 0007H \end{array}$$

$OF=0, CF=0, SF=0, ZF=0$

$0007H + 0001H = 0008H$

例. 双字加法运算, 设目的的操作数存放在 DX 和 AX 寄存器中, DX 放高位字,

源操作数存放在 BX 和 CX 寄存器中, BX 放高位字

指令执行前: $(DX) = 0020H$, $(AX) = 0F365H$, $(BX) = 0005H$, $(CX) = 0E024H$

$ADD\ AX, CX(1)$

$ADC\ DX, BX(2)$

(1) 执行后:

$$\begin{array}{r} 0F365H \\ +\ 0E024H \\ \hline 1D389H \end{array}$$

$OF=0, CF=1, SF=1, ZF=0$

(2) 执行后:

$$\begin{array}{r} 0020H \\ +\ 0005H \\ \hline 0025H \end{array}$$

$0025H + 0001H = 0026H$

$OF=0, CF=0, SF=0, ZF=0$

(2) 减法指令

1) 普通

SUB 目的操作数, 源操作数

$(目的) - (源) \rightarrow (目的)$

2) 带借位

SBB 目的操作数, 源操作数

$(目的) - (源) - CF \rightarrow (目的)$

3) 减1

DEC 目的操作数

$(目的) - 1 \rightarrow (目的)$

4) 求补

NEG 目的操作数

将目的操作数的每一位(包括符号位)求反, 末位加1, 结果送目的地址

我的理解: 对正数求对应负数的补码

对负数求以它作负数补码的对应的正数

其实就是“取相反数”

5) 比较

CMP 目的操作数, 源操作数

$(目的) - (源)$

结果只影响标志位而不送回目的地址给目的操作数

仅 DEC 指令不影响 CF 标志, 其余均对条件标志位有影响

减法指令对条件标志位的影响

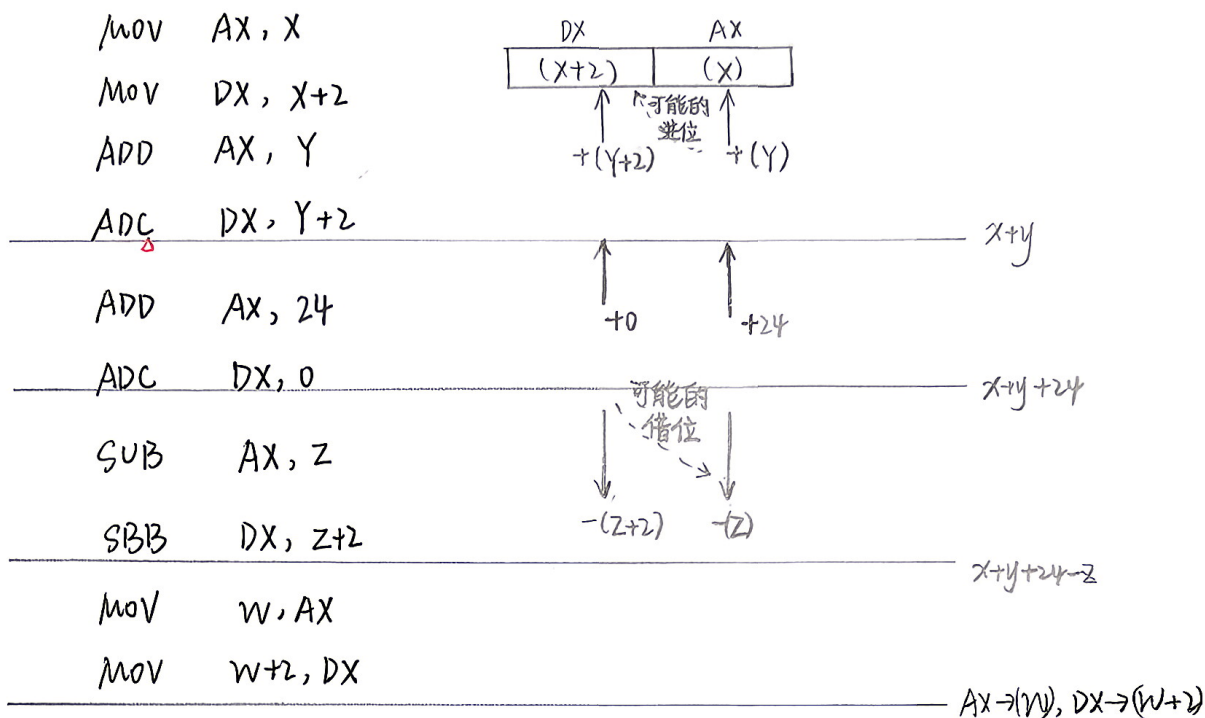
无符号数相减溢出 $\rightarrow$ CF (借位标志)

1 表示被减数的最高有效位有向高位的借位

有符号数相减溢出 $\rightarrow$ OF (溢出标志)

1 表示两操作数符号相反而结果符号与减数相同

例. x, y, z 均为双精度数, 分别存放在地址为 $X, X+2; Y, Y+2; Z, Z+2$ 的存储单元中, 用指令序列实现 $w \leftarrow x+y+24-z$, 并用 $W, W+2$ 单元存放 w



例. `MOV DX, 6780H`
`NEG DX`

解: 方法一: “`NEG DX`” 实际是 “ $0 - DX$ ” 操作

$$\begin{array}{r}
 6780H = 0110\ 0111\ 1000\ 0000\ B \\
 \quad \quad 0000\ 0000\ 0000\ 0000\ B \\
 - 0110\ 0111\ 1000\ 0000\ B \\
 \hline
 1001\ 1000\ 1000\ 0000\ B \quad \dots \rightarrow 9880H
 \end{array}$$

方法二:

$$\begin{array}{r}
 0110\ 0111\ 1000\ 0000\ B \\
 \downarrow \text{按位取反} \\
 1001\ 1000\ 0111\ 1111\ B \\
 \downarrow \text{末位加1} \\
 1001\ 1000\ 1000\ 0000\ B \quad \dots \rightarrow 9880H
 \end{array}$$

CF=1

例. 比较AL的内容数值大小

`CMP AL, 50`
 (AL) < 50
 低于则跳转 JB `Bellow`
`SUB AL, 50` (AL) ≥ 50 (AL) - 50 → AL
`INC AH` (AH) + 1 → AH
Bellow:
 ...

(3) 乘法指令

1) 无符号数

MUL 源操作数

2) 有符号数

IMUL 源操作数

若是字节数据相乘, 则(AL)与源操作数相乘得到的字数据存入AX中

若是字数据相乘, 则(AX)与源操作数相乘得到的双字数据, 高字存入DX, 低字存入AX中

隐含的
乘数寄存器

隐含的
乘积寄存器

- 源操作数不能是立即数
- 除CF, OF外, 对条件标志位没有定义:
这些标志在指令执行后是任意的, 不可预测的
对标志没有影响是指令的执行不改变标志的状态
- 乘法指令这样影响OF和CF标志

MUL指令: 若乘积的高一半(AH或DX)是0, 则OF=CF=0; 否则OF=CF=1

→ 用于检查字节相乘的结果是字节还是字、字相乘的结果是字还是双字

IMUL指令: 若乘积的高一半是低一半的符号扩展, 则OF=CF=0; 否则OF=CF=1

例.

无符号乘法

MOV AL, 0B4H → 180D

MOV BL, 11H → 17D

MUL BL $180 \times 17 = 3060D$

$3060D = 0BF4H$

AX = 0B4H × 11H = 0BF4H

OF = CF = 1

```

      1011 0100 B
    × 0001 0001 B
    -----
      1011 0100
     1011 0100
    -----
0000 1011 1110 0000 B
0  B F 4 H
```

有符号乘法

MOV AL, 0B4H → 10110100 → 10110011 → 01001100
-76D

MOV BL, 11H 17D

IMUL BL $-76 \times 17 = -1292D$
= -50CH

= - 01010000 1100 H

AX = 0B4H × 11H = FAF4H

负数的补码 OF = CF = 1 对应的正数

10110100 → 10110011 → 101001100

```

      0100 1100 B
    × 0001 0001 B
    -----
      0100 1100
     0100 1100
    -----
0000 0101 0000 1100 B(原)
      ↓
1101 0111 0100 B(补)
F A F 4 H
```

例. 编写程序段, 完成下面的计算公式, 并把所得的商和余数分别存入X和Y中
(其中: A, B, C, X和Y都是有符号的字变量)

$$(C - 120 + A * B) / C$$

解:

定义字变量

A	DW	?
B	DW	?
C	DW	?
X	DW	?
Y	DW	?

MOV AX, C

SUB AX, 120D ; 或 ADD AX, -120D

CWD ; 将带符号字扩展为双字, AX (16位) 扩展为 (DX, AX) (32位)
以便与 A * B 双字结果相加

MOV CX, DX } 把 DX, AX
MOV BX, AX } ↓
 CX, BX

调度寄存器, 把 DX, AX 腾出来

MOV AX, A

IMUL B

ADD ^{A*B} AX, BX ; 低16位相加

ADC _△ DX, CX ; 高16位相加

IDIV C

MOV X, AX 商

MOV Y, DX 余数

(4) 除法指令

1) 无符号数

DIV 源操作数
作除数

2) 有符号数

IDIV 源操作数

若是字节数据相除, 则 (AX) 与源操作数相除得到的商存入 AL, 余数存入 AH

若是字数据相除, 则 (DX, AX) 与源操作数相除得到的商存入 AX, 余数存入 DX

隐含的被除数寄存器

隐含的商寄存器 隐含的余数寄存器

- 源操作数不能是立即数
- 除法指令 DIV 和 IDIV 虽然对标志的影响未定义, 但可产生溢出

例. 无符号除法

MOV AX, 0400H

MOV BL, 0B4H

DIV BL

10110100 $\overline{) 1000000000}$
10110100
100110000
10110100
1111100B $\rightarrow 7CH$

AL = 05H, AH = 7CH

有符号除法 带符号除法的余数符号与 被除数符号相同

MOV AX, 0400H

MOV BL, 0B4H

IDIV BL

1001100 $\overline{) 1000000000}$
1001100
1101000
1001100
1110000
1001100
100100B $\rightarrow 24H$

AL = F3H, AH = 24H

当被除数远大于除数时, 所得的商可能超出范围, 若存放商的寄存器 AL/AX 不能表达就产生溢出 (8086 CPU 产生编号为 0 的内部中断: 除法错中断)

- 对 DIV 指令, 除数为 0 或字节除时商超过 8 位或字除时商超过 16 位, 发生除法溢出
- 对 IDIV 指令, 除数为 0 或字节除时商不在 $-128 \sim +127$ 或字除时商不在 $-32768 \sim 32767$, 发生除法溢出

(5) 符号扩展 (类型转换) 指令

1) 字节扩展为字指令 CBW

• 无操作数指令

AL $\rightarrow$ AX

• 隐含对 AL 或 AX 进行扩展

若 (AL) 最高有效位是 0, 则 (AH) = 00H

• 不影响条件标志位

若 (AL) 最高有效位是 1, 则 (AH) = FFH

2) 字扩展为双字指令 CWD

AX $\rightarrow$ (DX, AX)

若 (AX) 最高有效位是 0, 则 (DX) = 0000H

若 (AX) 最高有效位是 1, 则 (DX) = FFFFH

例. (AX) = 0BA45H

~~CBW ; (AX) = 0045H~~

CWD ; (DX) = 0FFFFH, (AX) = 0BA45H

应该有误: 不符合 "高位对低位符号扩展,

只加长数据位数而不改变数据大小"

的原则

例. MOV AL, 64H

CBW ; AX = 0064H

MOV AX, 0FF00H

负数补码
1111 1111 0000 0000H $\rightarrow$ 1111 1101 1111 1111H

CWD ; AX = FFFFFFF0H

↓
1000000 | 00000000H -256D

(6) 十进制调整指令

→ 用BCD码存十进制数

BCD (8421) 码: 用二进制编码的十进制数, 又称二-十进制数

压缩的BCD码: 用4位二进制数表示1位十进制数

$$(59)_{10} = (0101\ 1001)_{BCD}$$

非压缩的BCD码: 用8位二进制数表示1位十进制数

$$(59)_{10} = (0000\ 0101\ 0000\ 1001)_{BCD}$$

压缩的BCD码调整指令:

DAA 加法的十进制调整指令

DAS 减法的十进制调整指令

非压缩的BCD码调整指令:

AAA 加法的ASCII码调整指令

AAS 减法的ASCII码调整指令

AAM 乘法的ASCII码调整指令

AAD 除法的ASCII码调整指令

$$\begin{array}{r} 12 \\ + 23 \\ \hline 35 \end{array} \quad \begin{array}{r} 0001\ 0010 \\ + 0010\ 0011 \\ \hline 0011\ 0101 \end{array}$$

$$\begin{array}{r} 19 \\ + 08 \\ \hline 27 \end{array} \quad \begin{array}{r} 0001\ 1001 \\ 0000\ 1000 \\ \hline 0010\ 0001 \\ + AF=1 \uparrow 110 \\ \hline 0010\ 0111 \end{array}$$

1) 加法的十进制调整指令DAA

若AL寄存器中低4位大于9或辅助进位AF=1, 则AL=AL+6且AF=1

应先执行AL>=0A0H或CF=1, 则AL=AL+60H且CF=1, 同时SF、ZF、PF均有影响

以AL为目的操作数的
加/减法
指令

例: MOV AL, 68H ; (AL) = 68H = (0110 1000)_{BCD}

MOV BL, 28H ; (BL) = 28H = (0010 1000)_{BCD}

ADD AL, BL ; (AL) = (AL) + (BL) = 68H + 28H = $\begin{array}{r} 0110\ 1000 \\ + 0010\ 1000 \\ \hline 1001\ 0000 \end{array} = 90H$

DAA ; (AL) = (AL) + 6H = 96H

2) 减法的十进制调整指令DAS

若AL寄存器中低4位大于9或辅助进位=1, 则AL=AL-6且AF=1

若AL >= 0A0H或CF=1, 则AL=AL-60H且CF=1, 同时SF、ZF、PF均有影响

例:

MOV AL, 34H ; (AL) = 34H = (0011 0100)_{BCD}

MOV BL, 59H ; (BL) = 59H = (0101 1001)_{BCD}

SUB AL, BL ; (AL) = (AL) - (BL) = 34H - 59H = $\begin{array}{r} 0011\ 0100 \\ - 0101\ 1001 \\ \hline 1101\ 1011 \end{array} = 0DBH$

DAS ; (AL) = 0DBH - 6H - 60H = 75H

3) 加法的ASCII调整指令AAA

若AL寄存器中低4位大于9或辅助进位AF=1, 则AL=AL+6, AH=AH+1, AF=CF=1且AL高4位清零

否则: CF=AF=0, AL高4位清零

例:

MOV AX, 0608H ; (AX) = 0608H = (0000 0110 0000 1000)_{BCD}

MOV BL, 09H ; (BL) = 09H = (0000 1001)_{BCD}

ADD AL, BL ; (AL) = (AL) + (BL) = 08H + 09H =

AAA ; (AX) = 0707H

$$\begin{array}{r} 0000\ 1000 \\ + 0000\ 1001 \\ \hline 0001\ 0001 \\ \text{AL} \\ + 110 \end{array} = 11H$$

4) 减法的ASCII调整指令AAS

若AL寄存器中低4位大于9或辅助进位AF=1, 则AL=AL-6, AH=AH-1, AF=CF=1且AL高4位清零

否则: CF=AF=0, AL高4位清零, 其他标志位OF、PF、SF、ZF不确定

例:

MOV AX, 0608H ; (AX) = 0608H = (0000 0110 0000 1000)_{BCD}

MOV BL, 09H ; (BL) = 09H = (0000 1001)_{BCD}

SUB AL, BL ; (AL) = (AL) - (BL) = 08H - 09H = $\begin{array}{r} 0000\ 1000 \\ - 0000\ 1001 \\ \hline 1111\ 1111 \end{array} = FFH$

AAS ; (AX) = 0509H

1. 下列传送指令中正确的是

~~MOV [DI], [SI]~~ 不允许2个内存操作数 ~~MOV [DI+DI], AL~~ ^{基址} ^{变址}

~~MOV WORD PTR [BX], 0100H~~
将立即数写入[BX]指向的字内存单元(16位)

~~MOV AL, BX~~
8位 16位

2. 将字变量BUF的内容送入~~SI~~的正确指令是
^{目标寄存器}

~~MOV [SI], BUF~~ 将BUF内容写入SI指向的内存单元 ~~MOV SI, [BUF]~~ 将BUF对应的内存单元中的内容写入SI

~~MOV OFFSET BUF, SI~~ 目的操作数不能是常量 ~~MOV SI, OFFSET BUF~~ 将BUF的偏移地址送到SI
BUF的偏移地址

3. 设SP初值为3050H, 执行POP BX后, SP的值是 3052H.
16位

$$SP = 3050H + 2H = 3052H$$

4. 已知BX = 028AH, SI = 0294H, (DS: 051EH) = 5432H, 执行LEA BX, [BX+SI]后,
BX中内容是 051EH.

$$EA = BX + SI = 028AH + 0294H = 051EH$$

由题意, 要将有效地址EA存到BX寄存器当中

③ 逻辑运算类指令

逻辑指令 { 逻辑运算指令: AND OR NOT XOR TEST
与 或 非 异或 测试

移位操作指令: 算术移位、逻辑移位、双精度移位、循环移位、带进位的循环移位

(1) 逻辑运算指令

不影响标志位 { NOT (逻辑非): NOT 操作数 不可为立即数
 $\neg(\text{操作数}) \rightarrow (\text{操作数})$

CF OF SF ZF PF AF { AND (逻辑与): AND 目的操作数, 源操作数
0 0 无定义 复位(与0相与) $(\text{目的操作数}) \wedge (\text{源操作数}) \rightarrow (\text{目的操作数})$

依运算结果设置 { OR (逻辑或): OR 目的操作数, 源操作数
置位(与1相或) $(\text{目的操作数}) \vee (\text{源操作数}) \rightarrow (\text{目的操作数})$

XOR (异或): XOR 目的操作数, 源操作数
求反(与1相异或) $(\text{目的操作数}) \oplus (\text{源操作数}) \rightarrow (\text{目的操作数})$

TEST (测试): TEST 操作数1, 操作数2
 $(\text{操作数1}) \wedge (\text{操作数2})$

TEST 指令是对两操作数进行逻辑与运算, 结果不送回目的操作数, 仅按 AND 指令影响标志

TEST 与 AND 的关系正如 CMP 与 SUB 的关系

例.

MOV AL, 75H ; AL=75H

AND AL, 32H ; AL=30H CF=OF=0 SF=0 ZF=0 PF=1

OR AL, 71H ; AL=71H CF=OF=0 SF=0 ZF=0 PF=1

XOR AL, 0F1H ; AL=80H CF=OF=0 SF=1 ZF=0 PF=0

NOT AL ; AL=7FH 标志不变!

AND BL, 11110110B ; 将 BL 中 D3 位、D0 位清零

OR BL, 00001001B ; 将 BL 中 D3 位、D0 位置 1

XOR BL, 00001001B ; 将 BL 中 D3 位、D0 位取反

$$\begin{array}{r} 0111\ 0101 \\ \wedge\ 0011\ 0010 \\ \hline 0011\ 0000 \end{array}$$

$$\begin{array}{r} 0011\ 0000 \\ \vee\ 0111\ 0001 \\ \hline 0111\ 0001 \end{array}$$

$$\begin{array}{r} 1011\ 0110 \\ \oplus\ 0111\ 0001 \\ \hline 1100\ 0111 \end{array}$$

$$\begin{array}{r} 0111\ 1111 \\ \wedge\ 1111\ 0110 \\ \hline 0000\ 1001 \end{array}$$

(2) 移位指令

{ 逻辑移位 { 左移
 { 右移
{ 算术移位 { 左移
 { 右移

移位指令的第一个操作数是指定的被移位的操作数, 可以是寄存器或存储单元

第二个操作数表示移位位数, 为1表示移动1位, 为CL表示移动多位

按移入的位设置进位标志CF, 依移位后结果影响SF、ZF、PF CL寄存器值表示移位位移

1) 逻辑左移指令 SHL

同一条

SHL 寄存器|存储器, 1/CL

SHL 1位相当于无符号数 $\times 2$

最低位补0, 最高位进入CF

2) 逻辑右移指令 SHR

SHR 寄存器|存储器, 1/CL

SHR 1位相当于无符号数 $\div 2$

最高位补0, 最低位进入CF

3) 算术左移指令 SAL

SAL 寄存器|存储器, 1/CL

最低位补0, 最高位进入CF

4) 算术右移指令 SAR

SAR 寄存器|存储器, 1/CL

最高位不变, 最低位进入CF

例.

MOV DX, 6075H ; DX = 0110 0000 0111 0101 B

SHL DX, 1 ; DX = 1100 0000 1110 1010 B

SAR DX, 1 ; DX = 1110 0000 0111 0101 B

SHR DX, 1 ; DX = 0111 0000 0011 0101 B

MOV CL, 4 ; CL = 4, 标志不变

SAR DX, CL ; DX = 0000 0111 0000 0011 B

最后移出的位是1

例.

将AL寄存器中的无符号数乘以10, SF=0, ZF=0, PF=1

XOR AH, AH 或 SUB AH, AH ; 将AH清零, 同时使标志为CF=0
AND AH, 0

用AL的话
会溢出

SHL AX, 1 ; $AL \times 2 \rightarrow AX$

MOV BX, AX ; $AX = AL \times 2 \rightarrow BX$

SHL AX, 1 ; $AL \times 4 \rightarrow AX$

SHL AX, 1 ; $AL \times 8 \rightarrow AX$

ADD AX, BX ; $AL \times 8 + AL \times 2 = AL \times 10 \rightarrow AX$

5) 循环移位指令

要把从一端移出的位返回另一端形成循环

不带进位循环左移: ROL 寄存器 | 存储器, 1/CL

不带进位循环右移: ROR 寄存器 | 存储器, 1/CL

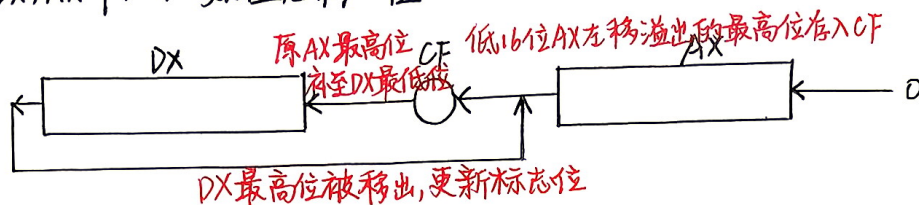
带进位循环左移: RCL 寄存器 | 存储器, 1/CL

带进位循环右移: RCR 寄存器 | 存储器, 1/CL

循环移位指令的操作数形式与移位指令相同,

按指令功能设置进位标志 CF 但不影响 SF、ZF、PF、AF 标志

例. 将 DX, AX 中 32 位数值左移一位



SHL AX, 1;

RCL DX, 1;

右移?

SHR DX, 1;

RCR AX, 1;

1. 设(CBL)=5, 要使结果(BL)=0AH, 应执行指令是

AND BL, 0FH
XOR BL, 0FH

ADD BL, 0FH
OR BL, 0FH

0101
↓
1010

取反

与全"1"异或 → 取反

与自己异或 → 清零

2. 将累加器AL清零并使进位标志CF清零, 错误的指令是

MOV ~~AL~~, 00H
XOR AL, AL

AND AL, 00H
SUB AL, AL

AND, OR, XOR, TEST 自带CF清零

3. 写出下列竖式的计算结果和状态标志的值, 并结合CF、OF解释运算结果

$$\begin{array}{r} 0000\ 1001 \\ + 0111\ 1100 \\ \hline 1000A\ 0101 \end{array}$$

CF=0, OF=1, SF=1, ZF=0, AF=1, PF=0

该运算在有符号数意义下发生溢出, 结果错误
该运算在无符号数意义下没有溢出, 结果正确